Inputs

Inputs to the `run_reopt` function can be provided in one of four formats:

1. a file path (string) to a JSON file,
2. a `Dict`,
3. using the `Scenario` struct, or
4. using the `REoptInputs` struct

Any one of these types can be passed to the run_reopt method as shown in Examples.

The first option is perhaps the most straightforward. For example, the minimum requirements for a JSON scenario file would look like:

```
{
    "Site": {
        "longitude": -118.1164613,
        "latitude": 34.5794343
    },
    "ElectricLoad": {
        "doe_reference_name": "MidriseApartment",
        "annual_kwh": 1000000.0
    },
    "ElectricTariff": {
        "urdb_label": "5ed6c1a15457a3367add15ae"
    }
}
```

The order of the keys does not matter. Note that this scenario does not include any energy generation technologies and therefore the results can be used as a baseline for comparison to scenarios that result in cost-optimal generation technologies (alternatively, a user could include a BAUScenario as shown in Examples).

To add PV to the analysis simply add a PV key with an empty dictionary (to use default values):

```
{
    "Site": {
        "longitude": -118.1164613,
        "latitude": 34.5794343
    },
    "ElectricLoad": {
        "doe_reference_name": "MidriseApartment",
        "annual_kwh": 1000000.0
    },
    "ElectricTariff": {
        "urdb_label": "5ed6c1a15457a3367add15ae"
    },
    "PV": {}
}
```

This scenario will consider the option to purchase a solar PV system to reduce energy costs, and if solar PV can reduce the energy costs then REopt will provide the optimal PV capacity (assuming perfect foresight!). See PV for all available input keys and default values for `PV`. To override a default value, simply specify a value for a given key. For example, the site under consideration might have some existing PV capacity to account for, which can be done by setting the `existing_kw` key to the appropriate value.

Scenario

The `Scenario` struct captures all of the possible user input keys (see Inputs for potential input formats). A Scenario struct will be automatically created if a `Dict` or file path are supplied to the run_reopt method. Alternatively, a user can create a `Scenario` struct and supply this to run_reopt.

`REopt.Scenario` — Type

```
Scenario(d::Dict; flex_hvac_from_json=false)
```

A Scenario struct can contain the following keys:

- Site (required)
- Financial (optional)

- ElectricTariff (required when `off_grid_flag=false`)
- ElectricLoad (required)
- PV (optional, can be Array)
- Wind (optional)
- ElectricStorage (optional)
- HotThermalStorage (optional)
- HighTempThermalStorage (optional)
- ColdThermalStorage (optional)
- ElectricStorage (optional)
- ElectricUtility (optional)
- Generator (optional)
- HeatingLoad (optional)
- CoolingLoad (optional)
- ExistingBoiler (optional)
- Boiler (optional)
- CHP (optional)
- FlexibleHVAC (optional)
- ExistingChiller (optional)
- AbsorptionChiller (optional)
- GHP (optional, can be Array)
- SteamTurbine (optional)
- ElectricHeater (optional)
- CST (optional)
- ASHPSpaceHeater (optional)
- ASHPWaterHeater (optional)

All values of `d` are expected to be `Dicts` except for `PV` and `GHP`, which can be either a `Dict` or `Dict[]` (for multiple PV arrays or GHP options).

> **Note**
>
> Set `flex_hvac_from_json=true` if `FlexibleHVAC` values were loaded in from JSON

(necessary to handle conversion of Vector of Vectors from JSON to a Matrix in Julia).

```
Scenario(fp::String)
```

Consruct Scenario from filepath `fp` to JSON with keys aligned with the `Scenario(d::Dict)` method.

BAUScenario

The Business-as-usual (BAU) inputs are automatically created based on the `BAUScenario` struct when a user supplies two `JuMP.Models` to `run_reopt()` (as shown in Examples). The outputs of the BAU scenario are used to calculate comparative results such as the `Financial` net present value (`npv`).

`REopt.BAUInputs` — Function

```
BAUInputs(p::REoptInputs)
```

The `BAUInputs` (REoptInputs for the Business As Usual scenario) are created based on the `BAUScenario`, which is in turn created based on the optimized-case `Scenario`.

The following assumptions are made for the BAU Inputs:

- `PV` and `Generator` `min_kw` and `max_kw` set to the `existing_kw` values
- `ExistingBoiler` and `ExistingChiller` # TODO
- All other generation and storage tech sizes set to zero
- Capital costs are assumed to be zero for existing `PV` and `Generator`
- O&M costs and all other tech inputs are assumed to be the same for existing `PV` and `Generator` as those specified for the optimized case
- Outage assumptions for deterministic vs stochastic # TODO

Settings

`REopt.Settings` — Type

Captures high-level inputs affecting the optimization.

`Settings` is an optional REopt input with the following keys and default values:

```
    time_steps_per_hour::Int = 1 # corresponds to the time steps per hour for use
    add_soc_incentive::Bool = true # when true, an incentive is added to the mode
    off_grid_flag::Bool = false # true if modeling an off-grid system, not connec
    include_climate_in_objective::Bool = false # true if climate costs of emissic
    include_health_in_objective::Bool = false # true if health costs of emissions
    solver_name::String = "HiGHS" # solver used to obtain a solution to model ins
```

Site

`REopt.Site` — Type

Inputs related to the physical location:

`Site` is a required REopt input with the following keys and default values:

```
    latitude::Real,
    longitude::Real,
    land_acres::Union{Real, Nothing} = nothing, # acres of land available for PV
    roof_squarefeet::Union{Real, Nothing} = nothing,
    min_resil_time_steps::Int=0, # The minimum number consecutive timesteps that
    mg_tech_sizes_equal_grid_sizes::Bool = true,
    sector::String = "commercial/industrial", # available options: ["commercial/i
    federal_sector_state::String = "",
    federal_procurement_type::String = "",
    CO2_emissions_reduction_min_fraction::Union{Float64, Nothing} = nothing,
    CO2_emissions_reduction_max_fraction::Union{Float64, Nothing} = nothing,
    bau_emissions_lb_CO2_per_year::Union{Float64, Nothing} = nothing, # Auto-popu
    bau_grid_emissions_lb_CO2_per_year::Union{Float64, Nothing} = nothing,
    renewable_electricity_min_fraction::Union{Float64, Nothing} = nothing,
    renewable_electricity_max_fraction::Union{Float64, Nothing} = nothing,
    include_grid_renewable_fraction_in_RE_constraints::Bool = false,
```

```
    include_exported_elec_emissions_in_total::Bool = true,
    include_exported_renewable_electricity_in_total::Bool = true,
    outdoor_air_temperature_degF::Union{Nothing, Array{<:Real,1}} = nothing,
    node::Int = 1,
```

ElectricLoad

REopt.ElectricLoad — Type

ElectricLoad is a required REopt input with the following keys and default values:

```
    loads_kw::Array{<:Real,1} = Real[],
    normalize_and_scale_load_profile_input::Bool = false,  # Takes loads_kw and r
    path_to_csv::String = "", # for csv containing loads_kw
    doe_reference_name::String = "",
    blended_doe_reference_names::Array{String, 1} = String[],
    blended_doe_reference_percents::Array{<:Real,1} = Real[], # Values should be
    year::Union{Int, Nothing} = doe_reference_name ≠ "" || blended_doe_reference_
    city::String = "",
    annual_kwh::Union{Real, Nothing} = nothing, # scales the load profile to this
    monthly_totals_kwh::Array{<:Real,1} = Real[], # scales the load profile to th
    monthly_peaks_kw::Array{<:Real,1} = Real[], # scales the load profile to thes
    critical_loads_kw::Union{Nothing, Array{Real,1}} = nothing,
    loads_kw_is_net::Bool = true, # set to true if loads_kw is already net of on-
    critical_loads_kw_is_net::Bool = false, # set to true if critical_loads_kw is
    critical_load_fraction::Real = off_grid_flag ? 1.0 : 0.5, # fractional input
    operating_reserve_required_fraction::Real = off_grid_flag ? 0.1 : 0.0, # if o
    min_load_met_annual_fraction::Real = off_grid_flag ? 0.99999 : 1.0 # if off g
```

Required inputs

Must provide either `loads_kw` or `path_to_csv` or [`doe_reference_name` and `city`] or `doe_reference_name` or [`blended_doe_reference_names` and `blended_doe_reference_percents`].

When only `doe_reference_name` is provided the `Site.latitude` and `Site.longitude` are used to look up the ASHRAE climate zone, which determines the appropriate DoE Commercial

Reference Building profile.

When using the [`doe_reference_name` and `city`] option, choose `city` from one of the cities used to represent the ASHRAE climate zones:

- Albuquerque
- Atlanta
- Baltimore
- Boulder
- Chicago
- Duluth
- Fairbanks
- Helena
- Houston
- LosAngeles
- LasVegas
- Miami
- Minneapolis
- Phoenix
- SanFrancisco
- Seattle

and `doe_reference_name` from:

- FastFoodRest
- FullServiceRest
- Hospital
- LargeHotel
- LargeOffice
- MediumOffice
- MidriseApartment
- Outpatient
- PrimarySchool

- RetailStore
- SecondarySchool
- SmallHotel
- SmallOffice
- StripMall
- Supermarket
- Warehouse
- FlatLoad # constant load year-round
- FlatLoad*245* # constant load all hours of the weekdays
- FlatLoad*167* # two 8-hour shifts for all days of the year; 6-10 a.m.
- FlatLoad*165* # two 8-hour shifts for the weekdays; 6-10 a.m.
- FlatLoad*87* # one 8-hour shift for all days of the year; 9 a.m.-5 p.m.
- FlatLoad*85* # one 8-hour shift for the weekdays; 9 a.m.-5 p.m.

Each `city` and `doe_reference_name` combination has a default `annual_kwh`, or you can provide your own `annual_kwh` or `monthly_totals_kwh` and the reference profile will be scaled appropriately.

Year

The ElectricLoad `year` is used in ElectricTariff to align rate schedules with weekdays/weekends. If providing your own `loads_kw`, ensure the `year` matches the year of your data. If utilizing `doe_reference_name` or `blended_doe_reference_names`, the default year of 2017 is used because these load profiles start on a Sunday.

Net Load and Load Scaling Considerations

If `loads_kw` is already net of on-site generation and you are modeling an existing generation source in REopt (e.g., PV), set `loads_kw_is_net=true` (default). If `loads_kw` is net and you are additionally using `normalize_and_scale_load_profile_input` along with `annual_kwh` or `monthly_totals_kwh`, the scaling will be applied to the net loads and the annual or monthly values you supply should also be net.

ElectricTariff

`REopt.ElectricTariff` — Method

`ElectricTariff` is a required REopt input for on-grid scenarios only (it cannot be supplied when `Settings.off_grid_flag` is true) with the following keys and default values:

```
    urdb_label::String="",
    urdb_response::Dict=Dict(), # Response JSON for URDB rates. Note: if creating
    urdb_utility_name::String="",
    urdb_rate_name::String="",
    urdb_metadata::Dict=Dict(), # Meta data about the URDB rate, from the URDB AP
    wholesale_rate::T1=nothing, # Price of electricity sold back to the grid in a
    export_rate_beyond_net_metering_limit::T2=nothing, # Price of electricity sol
    monthly_energy_rates::Array=[], # Array (length of 12) of blended energy rate
    monthly_demand_rates::Array=[], # Array (length of 12) of blended demand char
    blended_annual_energy_rate::S=nothing, # Annual blended energy rate [$ per kW
    blended_annual_demand_rate::R=nothing, # Average monthly demand charge [$ per
    add_monthly_rates_to_urdb_rate::Bool=false, # Set to 'true' to add the monthl
    tou_energy_rates_per_kwh::Array=[], # Time-of-use energy rates, provided by u
    add_tou_energy_rates_to_urdb_rate::Bool=false, # Set to 'true' to add the tou
    remove_tiers::Bool=false,
    demand_lookback_months::AbstractArray{Int64, 1}=Int64[], # Array of 12 binary
    demand_lookback_percent::Real=0.0, # Lookback percentage. Applies to either `
    demand_lookback_range::Int=0, # Number of months for which `demand_lookback_p
    coincident_peak_load_active_time_steps::Vector{Vector{Int64}}=[Int64[]], # Th
    coincident_peak_load_charge_per_kw::AbstractVector{<:Real}=Real[] # Optional
    ) where {
        T1 <: Union{Nothing, Real, Array{<:Real}},
        T2 <: Union{Nothing, Real, Array{<:Real}},
        S <: Union{Nothing, Real},
        R <: Union{Nothing, Real}
    }
```

Export Rates

There are three Export tiers and their associated export rates (negative cost values):

1. NEM (Net Energy Metering) - set to the energy rate (or tier with the lowest energy rate, if tiered)
2. WHL (Wholesale) - set to wholesale_rate
3. EXC (Excess, beyond NEM) - set to export*rate*beyond*net*metering_limit

Only one of NEM and Wholesale can be exported into due to the binary constraints. Excess can be exported into in the same time step as NEM.

Excess is meant to be combined with NEM: NEM export is limited to the total grid purchased energy in a year and some utilities offer a compensation mechanism for export beyond the site load. The Excess tier is not available with the Wholesale tier.

NEM input

The `NEM` boolean is determined by the `ElectricUtility.net_metering_limit_kw`. There is no need to pass in a `NEM` value.

Demand Lookback Inputs

Cannot use both `demand_lookback_months` and `demand_lookback_range` inputs, only one or the other. When using lookbacks, the peak demand in each month will be the greater of the peak kW in that month and the peak kW in the lookback months times the demand*lookback*percent.

Financial

`REopt.Financial` — Type

`Financial` is an optional REopt input with the following keys and default values:

```
    om_cost_escalation_rate_fraction::Real = get(get_sector_defaults(; sector=sec
    elec_cost_escalation_rate_fraction::Real = get(get_sector_defaults(; sector=s
    existing_boiler_fuel_cost_escalation_rate_fraction::Real = get(get_sector_def
    boiler_fuel_cost_escalation_rate_fraction::Real = get(get_sector_defaults(; s
```

```
    chp_fuel_cost_escalation_rate_fraction::Real = get(get_sector_defaults(; sect
    generator_fuel_cost_escalation_rate_fraction::Real = get(get_sector_defaults(
    offtaker_tax_rate_fraction::Real = get(get_sector_defaults(; sector=sector, f
    offtaker_discount_rate_fraction::Real = get(get_sector_defaults(; sector=sect
    third_party_ownership::Real = get(get_sector_defaults(; sector=sector, federa
    owner_tax_rate_fraction::Real = get(get_sector_defaults(; sector=sector, fede
    owner_discount_rate_fraction::Real = get(get_sector_defaults(; sector=sector,
    analysis_years::Int = 25,
    value_of_lost_load_per_kwh::Union{Array{R,1}, R} where R<:Real = 1.00, #only
    microgrid_upgrade_cost_fraction::Real = 0.0
    macrs_five_year::Array{Float64,1} = [0.2, 0.32, 0.192, 0.1152, 0.1152, 0.0576
    macrs_seven_year::Array{Float64,1} = [0.1429, 0.2449, 0.1749, 0.1249, 0.0893,
    offgrid_other_capital_costs::Real = 0.0, # only applicable when `off_grid_fla
    offgrid_other_annual_costs::Real = 0.0 # only applicable when `off_grid_flag`
    min_initial_capital_costs_before_incentives::Union{Nothing,Real} = nothing #
    max_initial_capital_costs_before_incentives::Union{Nothing,Real} = nothing #
    # Emissions cost inputs
    CO2_cost_per_tonne::Real = 51.0,
    CO2_cost_escalation_rate_fraction::Real = 0.042173,
    NOx_grid_cost_per_tonne::Union{Nothing,Real} = nothing,
    SO2_grid_cost_per_tonne::Union{Nothing,Real} = nothing,
    PM25_grid_cost_per_tonne::Union{Nothing,Real} = nothing,
    NOx_onsite_fuelburn_cost_per_tonne::Union{Nothing,Real} = nothing, # Default
    SO2_onsite_fuelburn_cost_per_tonne::Union{Nothing,Real} = nothing, # Default
    PM25_onsite_fuelburn_cost_per_tonne::Union{Nothing,Real} = nothing, # Default
    NOx_cost_escalation_rate_fraction::Union{Nothing,Real} = nothing, # Default c
    SO2_cost_escalation_rate_fraction::Union{Nothing,Real} = nothing, # Default c
    PM25_cost_escalation_rate_fraction::Union{Nothing,Real} = nothing # Default c
```

Third party financing

When `third_party_ownership` is `false` the offtaker's discount and tax percentages are used throughout the model:

```
if !third_party_ownership
    owner_tax_rate_fraction = offtaker_tax_rate_fraction
    owner_discount_rate_fraction = offtaker_discount_rate_fraction
end
```

ElectricUtility

REopt.ElectricUtility — Type

ElectricUtility is an optional REopt input with the following keys and default values:

```
    net_metering_limit_kw::Real = 0, # Upper limit on the total capacity of techn
    interconnection_limit_kw::Real = 1.0e9, # Limit on total electric system capa
    allow_simultaneous_export_import::Bool = true,  # if true the site has two me

    # Single Outage Modeling Inputs (Outage Modeling Option 1):
    outage_start_time_step::Int=0,  # for modeling a single outage, with critical
    outage_end_time_step::Int=0,  # ... utility production_factor = 0 during the

    # Multiple Outage Modeling Inputs (Outage Modeling Option 2):
    # minimax the expected outage cost, with max taken over outage start time, ex
    outage_start_time_steps::Array{Int,1}=Int[],  # we minimize the maximum outag
    outage_durations::Array{Int,1}=Int[],  # One-to-one with outage_probabilities
    outage_probabilities::Array{R,1} where R<:Real = [1.0],

    ### Cambium Emissions and Clean Energy Inputs ###
    cambium_scenario::String = "Mid-case", # Cambium Scenario for evolution of el
        ## Options: ["Mid-case", "Low renewable energy cost",   "High renewable e
    cambium_location_type::String =  "GEA Regions 2023", # Geographic boundary at
    cambium_start_year::Int = 2025, # First year of operation of system. Emission
    cambium_levelization_years::Int = analysis_years, # Expected lifetime or anal
    cambium_grid_level::String = "enduse", # Options: ["enduse", "busbar"]. Busba

    ### Grid Climate Emissions Inputs ###
    # Climate Option 1 (Default): Use levelized emissions data from NLR's Cambium
    cambium_co2_metric::String = "lrmer_co2e", # Emissions metric used. Default:

    # Climate Option 2: Use CO2 emissions data from the EPA's AVERT based on the
    co2_from_avert::Bool = false, # Default is to use Cambium data for CO2 grid e

    # Climate Option 3: Provide your own custom emissions factors for CO2 and spe
    emissions_factor_series_lb_CO2_per_kwh::Union{Real,Array{<:Real,1}} = Float64

    # Used with Climate Options 2 or 3: Annual percent decrease in CO2 emissions
```

```
    emissions_factor_CO2_decrease_fraction::Union{Nothing, Real} = co2_from_avert

    ### Grid Health Emissions Inputs ###
    # Health Option 1 (Default): Use health emissions data from the EPA's AVERT b
    avert_emissions_region::String = "", # AVERT emissions region. Default is bas

    # Health Option 2: Provide your own custom emissions factors for health emiss
    emissions_factor_series_lb_NOx_per_kwh::Union{Real,Array{<:Real,1}} = Float64
    emissions_factor_series_lb_SO2_per_kwh::Union{Real,Array{<:Real,1}} = Float64
    emissions_factor_series_lb_PM25_per_kwh::Union{Real,Array{<:Real,1}} = Float6

    # Used with Health Options 1 or 2: Annual percent decrease in health emission
    emissions_factor_NOx_decrease_fraction::Real = EMISSIONS_DECREASE_DEFAULTS["N
    emissions_factor_SO2_decrease_fraction::Real = EMISSIONS_DECREASE_DEFAULTS["S
    emissions_factor_PM25_decrease_fraction::Real = EMISSIONS_DECREASE_DEFAULTS['

    ### Grid Clean Energy Fraction Inputs ###
    cambium_cef_metric::String = "cef_load", # Options = ["cef_load", "cef_gen"]
    renewable_energy_fraction_series::Union{Real,Array{<:Real,1}} = Float64[], #
```

Outage modeling

Indexing

Outage indexing begins at 1 (not 0) and the outage is inclusive of the outage end time step. For instance, to model a 3-hour outage from 12AM to 3AM on Jan 1, outage*start*time*step = 1 and outage*end*time*step = 3. To model a 1-hour outage from 6AM to 7AM on Jan 1, outage*start*time*step = 7 and outage*end*time*step = 7.

Can use either singular or multiple outage modeling inputs, not both

Cannot supply singular outage*start(or end)*time*step and multiple outage*start*time*steps. Must use one or the other.

Using min*resil*time_steps to ensure critical

load is met

With multiple outage modeling, the model will choose to meet the critical loads only as cost-optimal. This trade-off depends on cost of not meeting load (see `Financial | value_of_lost_load_per_kwh`) and the costs of meeting load, such as microgrid upgrade cost (see `Financial | microgrid_upgrade_cost_fraction`), fuel costs, and additional DER capacity. To ensure that REopt recommends a system that can meet critical loads during a defined outage period, specify this duration using `Site | min_resil_time_steps`.

Outage costs will be included in NPV and LCC

Note that when using multiple outage modeling, the expected outage cost will be included in the net present value and lifecycle cost calculations (for both the BAU and optimized case). You can set `Financial | value_of_lost_load_per_kwh` to 0 to ignore these costs. However, doing so will remove incentive for the model to meet critical loads during outages, and you should therefore consider also specifying `Site | min_resil_time_steps`. You can alternatively post-process results to remove `lifecycle_outage_cost` from the NPV and LCCs.

Outages, Emissions, and Renewable Energy Calculations

If a single deterministic outage is modeled using outage*start*time*step and outage*end*time*step, emissions and renewable energy percentage calculations and constraints will factor in this outage. If stochastic outages are modeled using outage*start*time*steps*, *outage*durations, and outage_probabilities, emissions and renewable energy percentage calculations and constraints will not consider outages.

MPC vs. Non-MPC

This constructor is intended to be used with latitude/longitude arguments provided for the non-MPC case and without latitude/longitude arguments provided for the MPC case.

Climate and Health Emissions and Grid Clean Energy Modeling

Climate and health-related emissions from grid electricity come from two different data sources and have different REopt inputs as described below.

Climate Emissions

- For sites in the contiguous United States (CONUS):
  - Default climate-related emissions factors come from NLR's Cambium database (Current version: 2022)
    - By default, REopt uses *levelized long-run marginal emission rates for CO2-equivalent (CO2e) emissions* for the region in which the site is located. By default, the emissions rates are levelized over the analysis period (e.g., from 2025 through 2049 for a 25-year analysis)
    - The inputs to the Cambium API request can be modified by the user based on emissions accounting needs (e.g., can change "lifetime" to 1 to analyze a single year's emissions)
    - Note for analysis periods extending beyond 2050: Values beyond 2050 are estimated with the 2050 values. Analysts are advised to use caution when selecting values that place significant weight on 2050 (e.g., greater than 50%)
  - Users can alternatively choose to use emissions factors from the EPA's AVERT by setting `co2_from_avert` to `true`
- For Alaska and HI: Grid CO2e emissions rates come from the eGRID database. These are single values repeated throughout the year. The default annual `emissions_factor_CO2_decrease_fraction` will be applied to this rate to account for future greening of the grid.
- For sites outside of the United States: REopt does not have default grid emissions rates for sites outside of the U.S. For these sites, users must supply custom emissions factor series (`emissions_factor_series_lb_CO2_per_kwh`) and projected emissions decreases (`emissions_factor_CO2_decrease_fraction`).

Health Emissions

- For sites in CONUS: health-related emissions factors (PM2.5, SO2, and NOx) come from the EPA's AVERT database.

- For Alaska and HI: Grid health emissions rates come from the eGRID database. These are single values repeated throughout the year. The default annual `emissions_factor_XXX_decrease_fraction` will be applied to this rate to account for future greening of the grid.
- The default `avert_emissions_region` input is determined by the site's latitude and longitude.

Alternatively, you may input the desired AVERT `avert_emissions_region`, which must be one of: ["California", "Central", "Florida", "Mid-Atlantic", "Midwest", "Carolinas", "New England","Northwest", "New York", "Rocky Mountains", "Southeast", "Southwest", "Tennessee", "Texas", "Alaska", "Hawaii (except Oahu)", "Hawaii (Oahu)"]

- For sites outside of the United States: REopt does not have default grid emissions rates for sites outside of the U.S. For these sites, users must supply custom emissions factor series (e.g., `emissions_factor_series_lb_NOx_per_kwh`) and projected emissions decreases (e.g., `emissions_factor_NOx_decrease_fraction`).

Grid Clean Energy Fraction

- For sites in CONUS:
  - Default clean energy fraction data comes from NLR's Cambium database (Current version: 2022)
    - By default, REopt uses *clean energy fraction* for the region in which the site is located.
- For sites outside of CONUS: REopt does not have default grid clean energy fraction data. Users must supply a custom `renewable_energy_fraction_series`

PV

`REopt.PV` — Type

`PV` is an optional REopt input with the following keys and default values:

```
    array_type::Int=1, # PV Watts array type (0: Ground Mount Fixed (Open Rack);
```

```
    tilt::Real = (array_type == 0 || array_type == 1) ? 20 : 0, # tilt = 20 for f
    module_type::Int=0, # PV module type (0: Standard; 1: Premium; 2: Thin Film)
    losses::Real=0.14, # System losses
    azimuth::Real = latitude≥0 ? 180 : 0, # set azimuth to zero for southern hemi
    gcr::Real=0.4,  # Ground coverage ratio
    radius::Int=0, # Radius, in miles, to use when searching for the closest clim
    name::String="PV", # for use with multiple pvs
    location::String="both", # one of ["roof", "ground", "both"]
    existing_kw::Real=0,
    min_kw::Real=0,
    max_kw::Real=1.0e9, # max new DC capacity (beyond existing_kw)
    installed_cost_per_kw::Union{Real, AbstractVector{<:Real}} = Float64[], # def
    om_cost_per_kw::Real=18.0,
    degradation_fraction::Real=0.005,
    macrs_option_years::Int = get(get_sector_defaults(; sector=sector, federal_pr
    macrs_bonus_fraction::Real = get(get_sector_defaults(; sector=sector, federal
    macrs_itc_reduction::Real = 0.5,
    kw_per_square_foot::Real=0.01,
    acres_per_kw::Real=6e-3,
    inv_eff::Real=0.96,
    dc_ac_ratio::Real=1.2,
    production_factor_series::Union{Nothing, Array{<:Real,1}} = nothing, # Option
    federal_itc_fraction::Real = get(get_sector_defaults(; sector=sector, federal
    federal_rebate_per_kw::Real = 0.0,
    state_ibi_fraction::Real = 0.0,
    state_ibi_max::Real = 1.0e10,
    state_rebate_per_kw::Real = 0.0,
    state_rebate_max::Real = 1.0e10,
    utility_ibi_fraction::Real = 0.0,
    utility_ibi_max::Real = 1.0e10,
    utility_rebate_per_kw::Real = 0.0,
    utility_rebate_max::Real = 1.0e10,
    production_incentive_per_kwh::Float64 = 0.0 # revenue from production incenti
    production_incentive_max_benefit::Float64 = 1.0e9 # maximum allowable annual
    production_incentive_years::Int = 1 # number of year in which production ince
    production_incentive_max_kw::Float64 = 1.0e9 # maximum allowable system size
    can_net_meter::Bool = off_grid_flag ? false : true,
    can_wholesale::Bool = off_grid_flag ? false : true,
    can_export_beyond_nem_limit::Bool = off_grid_flag ? false : true,
    can_curtail::Bool = true,
```

```
    operating_reserve_required_fraction::Real = off_grid_flag ? 0.25 : 0.0, # if
    size_class::Union{Int, Nothing} = nothing, # Size class for cost curve select
    tech_sizes_for_cost_curve::AbstractVector = Float64[], # System sizes for det
    use_detailed_cost_curve::Bool = false, # Use detailed cost curve instead of a
    electric_load_annual_kwh::Real = 0.0, # Annual electric load (kWh) for size c
    site_land_acres::Union{Real, Nothing} = nothing,  # site.land_acres to determ
    site_roof_squarefeet::Union{Real, Nothing} = nothing  # site.roof_squarefeet
```

Multiple PV types

Multiple PV types can be considered by providing an array of PV inputs. See example in `src/test/scenarios/multiple_pvs.json`

PV tilt and aziumth

If `tilt` is not provided, then it is set to the absolute value of `Site.latitude` for ground-mount systems and is set to 10 degrees for rooftop systems. If `azimuth` is not provided, then it is set to 180 if the site is in the northern hemisphere and 0 if in the southern hemisphere.

Cost curves and size classes

When using 'use*detailed*cost*curve' is set to* `true`*, and providing specific values for `tech*sizes*for*cost*curve and*installed*cost*per*kw* , `both*tech*sizes*for*cost*curve*and*installed*cost*per*kw` must have the same length. Size class is automatically determined based on average load if not specified, which affects default costs. Ground-mount('array*type' = 0,2,3,4) systems have different cost structures than rooftop ('array_type' = 1) systems when using default values.

Wind

`REopt.Wind` — Type

`Wind` is an optional REopt input with the following keys and default values:

```
    min_kw = 0.0,
    max_kw = 1.0e9,
    installed_cost_per_kw = nothing,
    om_cost_per_kw = 42.0,
    production_factor_series = nothing, # Optional user-defined production facto
    size_class = "",
    wind_meters_per_sec = [],
    wind_direction_degrees = [],
    temperature_celsius = [],
    pressure_atmospheres = [],
    acres_per_kw = 0.03, # assuming a power density of 30 acres per MW for turbi
    macrs_option_years = get(get_sector_defaults(; sector=sector, federal_procure
    macrs_bonus_fraction = get(get_sector_defaults(; sector=sector, federal_procu
    macrs_itc_reduction = 0.5,
    federal_itc_fraction = get(get_sector_defaults(; sector=sector, federal_procu
    federal_rebate_per_kw = 0.0,
    state_ibi_fraction = 0.0,
    state_ibi_max = 1.0e10,
    state_rebate_per_kw = 0.0,
    state_rebate_max = 1.0e10,
    utility_ibi_fraction = 0.0,
    utility_ibi_max = 1.0e10,
    utility_rebate_per_kw = 0.0,
    utility_rebate_max = 1.0e10,
    production_incentive_per_kwh::Float64 = 0.0 # revenue from production incenti
    production_incentive_max_benefit::Float64 = 1.0e9 # maximum allowable annual
    production_incentive_years::Int = 1 # number of year in which production ince
    production_incentive_max_kw::Float64 = 1.0e9 # maximum allowable system size
    can_net_meter = true,
    can_wholesale = true,
    can_export_beyond_nem_limit = true
    operating_reserve_required_fraction::Real = off_grid_flag ? 0.50 : 0.0, # Onl
```

Default assumptions

`size_class` must be one of ["residential", "commercial", "medium", "large"]. If `size_class` is not provided then it is determined based on the average electric load.

If no `installed_cost_per_kw` is provided then it is determined from:

```
size_class_to_installed_cost = Dict(
    "residential"=> 7692.0,
    "commercial"=> 5776.0,
    "medium"=> 3807.0,
    "large"=> 2896.0
)
```

If the `production_factor_series` is not provided then NLR's System Advisor Model (SAM) is used to get the wind turbine production factor.

Wind resource value inputs

Wind resource values are optional (i.e., `wind_meters_per_sec`, `wind_direction_degrees`, `temperature_celsius`, and `pressure_atmospheres`). If not provided then the resource values are downloaded from NLR's Wind Toolkit. These values are passed to SAM to get the turbine production factor.

Wind sizing and land constraint

Wind size is constrained by Site.land*acres, assuming a power density of Wind.acres*per_kw for turbine sizes above 1.5 MW (default assumption of 30 acres per MW). If the turbine size recommended is smaller than 1.5 MW, the input for land available will not constrain the system size. If the the land available constrains the system size to less than 1.5 MW, the system will be capped at 1.5 MW (i.e., turbines < 1.5 MW are not subject to the acres/kW limit).

ElectricStorage

`REopt.ElectricStorageDefaults` — Type

`ElectricStorage` is an optional optional REopt input with the following keys and default values:

```
    min_kw::Real = 0.0
    max_kw::Real = 1.0e4
```

```
    min_kwh::Real = 0.0
    max_kwh::Real = 1.0e6
    internal_efficiency_fraction::Float64 = 0.975
    inverter_efficiency_fraction::Float64 = 0.96
    rectifier_efficiency_fraction::Float64 = 0.96
    soc_min_fraction::Float64 = 0.2
    soc_min_applies_during_outages::Bool = false
    soc_init_fraction::Float64 = off_grid_flag ? 1.0 : 0.5
    can_grid_charge::Bool = off_grid_flag ? false : true
    installed_cost_per_kw::Real = 968.0 # Cost of power components (e.g., inverte
    installed_cost_per_kwh::Real = 253.0 # Cost of energy components (e.g., batte
    installed_cost_constant::Real = 222115.0 # "+c" constant cost that is added t
    replace_cost_per_kw::Real = 0.0
    replace_cost_per_kwh::Real = 0.0
    replace_cost_constant::Real = 0.0
    inverter_replacement_year::Int = 10
    battery_replacement_year::Int = 10
    cost_constant_replacement_year::Int = 10
    om_cost_fraction_of_installed_cost::Float64 = 0.025 # Annual O&M cost as a fr
    macrs_option_years::Int = 5 #Note: default may change if Site.sector is not '
    macrs_bonus_fraction::Float64 = 1.0 #Note: default may change if Site.sector
    macrs_itc_reduction::Float64 = 0.5
    total_itc_fraction::Float64 = 0.3 #Note: default may change if Site.sector is
    total_rebate_per_kw::Real = 0.0
    total_rebate_per_kwh::Real = 0.0
    charge_efficiency::Float64 = rectifier_efficiency_fraction * internal_efficie
    discharge_efficiency::Float64 = inverter_efficiency_fraction * internal_effic
    grid_charge_efficiency::Float64 = can_grid_charge ? charge_efficiency : 0.0
    model_degradation::Bool = false
    degradation::Dict = Dict()
    minimum_avg_soc_fraction::Float64 = 0.0
    optimize_soc_init_fraction::Bool = false # If true, soc_init_fraction will no
    min_duration_hours::Real = 0.0 # Minimum amount of time storage can discharge
    max_duration_hours::Real = 100000.0 # Maximum amount of time storage can disc
```

REopt.Degradation — Type

```
Degradation
```

Inputs used when `ElectricStorage.model_degradation` is `true`:

```
Base.@kwdef mutable struct Degradation
    calendar_fade_coefficient::Real = 1.16E-03
    cycle_fade_coefficient::Vector{<:Real} = [2.46E-05]
    cycle_fade_fraction::Vector{<:Real} = [1.0]
    time_exponent::Real = 0.428
    installed_cost_per_kwh_declination_rate::Real = 0.05
    maintenance_strategy::String = "augmentation"  # one of ["augmentation", "rep
    maintenance_cost_per_kwh::Vector{<:Real} = Real[]
end
```

None of the above values are required. If `ElectricStorage.model_degradation` is `true` then the defaults above are used. If the `maintenance_cost_per_kwh` is not provided then it is determined using the `ElectricStorage.installed_cost_per_kwh` and the `installed_cost_per_kwh_declination_rate` along with a present worth factor f to account for the present cost of buying a battery in the future. The present worth factor for each day is:

$$f(day) = \frac{(1-r_g)^{\frac{day}{365}}}{(1+r_d)^{\frac{day}{365}}}$$

where r_g = `installed_cost_per_kwh_declination_rate` and r_d = `p.s.financial.owner_discount_rate_fraction`.

Note this day-specific calculation of the present-worth factor accumulates differently from the annually updated discount rate for other net-present value calculations in REopt, and has a higher effective discount rate as a result. The present worth factor is used in the same manner irrespective of the `maintenance_strategy`.

Warn

When modeling degradation the following ElectricStorage inputs are not used:

- `replace_cost_per_kwh`
- `battery_replacement_year`
- `replace_cost_constant`
- `cost_constant_replacement_year`

They are replaced by the `maintenance_cost_per_kwh` vector. Inverter replacement costs and inverter replacement year should still be used to model scheduled replacement of inverter.

Note

When providing the `maintenance_cost_per_kwh` it must have a length equal to `Financial.analysis_years*365-1`.

Battery State Of Health

The state of health [`SOH`] is a linear function of the daily average state of charge [`Eavg`] and the daily equivalent full cycles [`EFC`]. The initial `SOH` is set to the optimal battery energy capacity (in kWh). The evolution of the `SOH` beyond the first day is:

$$SOH[d] = SOH[d-1] - h\left(\tfrac{1}{2}k_{cal}Eavg[d-1]/\sqrt{d} + k_{cyc}EFC[d-1] \quad \forall d \in \{2\ldots D\}\right)$$

where:

- k_{cal} is the `calendar_fade_coefficient`
- k_{cyc} is the `cycle_fade_coefficient`
- h is the hours per time step
- D is the total number of days, 365 * `analysis_years`

The `SOH` is used to determine the maintence cost of the storage system, which depends on the `maintenance_strategy`.

Note

Battery degradation parameters are from based on laboratory aging data, and are expected to be reasonable only within the range of conditions tested. Battery lifetime can vary widely from these estimates based on battery use and system design. Battery cost estimates are based on domain expertise and published guidelines and are not to be taken as an indicator of real system costs.

Augmentation Maintenance Strategy

The augmentation maintenance strategy assumes that the battery energy capacity is maintained by replacing degraded cells daily in terms of cost. Using the definition of the `SOH` above the maintenance cost is:

$$C_{\text{aug}} = \sum_{d \in \{2 \ldots D\}} C_{\text{install}} f(day) \left(SOH[d-1] - SOH[d]\right)$$

where

- $f(day)$ is the present worth factor of battery degradation costs as described above;
- C_{install} is the `ElectricStorage.installed_cost_per_kwh`; and
- $SOH[d-1] - SOH[d]$ is the incremental amount of battery capacity lost in a day.

The C_{aug} is added to the objective function to be minimized with all other costs.

Replacement Maintenance Strategy

Modeling the replacement maintenance strategy is more complex than the augmentation strategy. Effectively the replacement strategy says that the battery has to be replaced once the `SOH` drops below 80% of the optimal, purchased capacity. It is possible that multiple replacements (at same replacement frequency) could be required under this strategy.

> **Warn**
>
> The "replacement" maintenance strategy requires integer decision variables. Some solvers are slow with integer decision variables.

The replacement strategy cost is:

$$C_{\text{repl}} = B_{\text{kWh}} N_{\text{repl}} f(d_{80}) C_{\text{install}}$$

where:

- B_{kWh} is the optimal battery capacity (`ElectricStorage.size_kwh` in the results dictionary);
- N_{repl} is the number of battery replacments required (a function of the month in which the `SOH` falls below 80% of original capacity);

- $f(d_{80})$ is the present worth factor at approximately the 15th day of the month in which the `SOH` falls below 80% of original capacity;
- C_{install} is the `ElectricStorage.installed_cost_per_kwh`.

The C_{repl} is added to the objective function to be minimized with all other costs.

Battery residual value

Since the battery can be replaced one-to-many times under this strategy, battery residual value captures the $ value of remaining battery life at end of analysis period. For example if replacement happens in month 145, then assuming 25 year analysis period there will be 2 replacements (months 145 and 290). The last battery which was placed in service during month 290 only serves for 10 months (i.e. 6.89% of its expected life assuming 145 month replacement frequecy). In this case, the battery has 93.1% of residual life remaining as useful life left after analysis period ends. A residual value cost vector is created to hold this value for all months. Residual value is calculated as:

$C_{\text{residual}} = Rf(d_{\text{last}})C_{\text{install}}$ where:

- R is the `residual_factor` which determines portion of battery life remaining at the end of the analysis period;
- $f(d_{\text{last}})$ is the present worth factor at approximately the 15th day of the last month in the analysis period;
- C_{install} is the `ElectricStorage.installed_cost_per_kwh`.

The C_{residual} is added to the objective function to be minimized with all other costs.

Example of inputs

The following shows how one would use the degradation model in REopt via the Scenario inputs:

```
{
    ...
    "ElectricStorage": {
        "installed_cost_per_kwh": 390,
        ...
        "model_degradation": true,
        "degradation": {
```

```
            "calendar_fade_coefficient": 1.16E-03,
            "cycle_fade_coefficient": [2.46E-05],
            "cycle_fade_fraction": [1.0],
            "time_exponent": 0.428
            "installed_cost_per_kwh_declination_rate": 0.05,
            "maintenance_strategy": "replacement",
            ...
        }
    },
    ...
}
```

Note that not all of the above inputs are necessary. When not providing `calendar_fade_coefficient` for example the default value will be used.

Generator

`REopt.Generator` — Type

`Generator` is an optional REopt input with the following keys and default values:

```
    only_runs_during_grid_outage::Bool = true,
    existing_kw::Real = 0,
    min_kw::Real = 0,
    max_kw::Real = 1.0e6,
    installed_cost_per_kw::Real = off_grid_flag ? 880 : only_runs_during_grid_out
    om_cost_per_kw::Real = off_grid_flag ? 10.0 : 20.0,
    om_cost_per_kwh::Real = 0.0,
    fuel_cost_per_gallon::Real = 2.25,
    electric_efficiency_full_load::Real = 0.322,
    electric_efficiency_half_load::Real = electric_efficiency_full_load,
    fuel_avail_gal::Real = 1.0e9,
    fuel_higher_heating_value_kwh_per_gal::Real = 40.7,
    min_turn_down_fraction::Real = off_grid_flag ? 0.15 : 0.0,
    sells_energy_back_to_grid::Bool = false,
    can_net_meter::Bool = false,
    can_wholesale::Bool = false,
```

```
    can_export_beyond_nem_limit = false,
    can_curtail::Bool = false,
    macrs_option_years::Int = 0,
    macrs_bonus_fraction::Real = 0.0,
    macrs_itc_reduction::Real = 0.0,
    federal_itc_fraction::Real = 0.0,
    federal_rebate_per_kw::Real = 0.0,
    state_ibi_fraction::Real = 0.0,
    state_ibi_max::Real = 1.0e10,
    state_rebate_per_kw::Real = 0.0,
    state_rebate_max::Real = 1.0e10,
    utility_ibi_fraction::Real = 0.0,
    utility_ibi_max::Real = 1.0e10,
    utility_rebate_per_kw::Real = 0.0,
    utility_rebate_max::Real = 1.0e10,
    production_incentive_per_kwh::Float64 = 0.0 # revenue from production incenti
    production_incentive_max_benefit::Float64 = 1.0e9 # maximum allowable annual
    production_incentive_years::Int = 0 # number of year in which production ince
    production_incentive_max_kw::Float64 = 1.0e9 # maximum allowable system size
    fuel_renewable_energy_fraction::Real = 0.0,
    emissions_factor_lb_CO2_per_gal::Real = 22.58, # CO2e
    emissions_factor_lb_NOx_per_gal::Real = 0.0775544,
    emissions_factor_lb_SO2_per_gal::Real = 0.040020476,
    emissions_factor_lb_PM25_per_gal::Real = 0.0,
    replacement_year::Int = off_grid_flag ? 10 : analysis_years, # Project year i
    replace_cost_per_kw::Real = off_grid_flag ? installed_cost_per_kw : 0.0 # Per
```

Replacement costs

Generator replacement costs will not be considered if `Generator.replacement_year >= Financial.analysis_years`.

ExistingBoiler

`REopt.ExistingBoiler` — Type

`ExistingBoiler` is an optional REopt input with the following keys and default values:

```
    max_heat_demand_kw::Real=0, # Auto-populated based on SpaceHeatingLoad and Do
    production_type::String = "hot_water", # Can be "steam" or "hot_water"
    max_thermal_factor_on_peak_load::Real = 1.25,
    installed_cost_per_mmbtu_per_hour::Real = 0.0  # Represents needed CapEx in B
    installed_cost_dollars::Real = 0.0  # Represents needed CapEx in BAU, assumir
    efficiency::Real = NaN, # Existing boiler system efficiency - conversion of f
    fuel_cost_per_mmbtu::Union{<:Real, AbstractVector{<:Real}} = [], # REQUIRED.
    fuel_type::String = "natural_gas", # "restrict_to": ["natural_gas", "landfill
    can_supply_steam_turbine::Bool = false,
    retire_in_optimal::Bool = false,  # Do NOT use in the optimal case (still use
    fuel_renewable_energy_fraction::Real = get(FUEL_DEFAULTS["fuel_renewable_ener
    emissions_factor_lb_CO2_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_factor
    emissions_factor_lb_NOx_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_factor
    emissions_factor_lb_SO2_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_factor
    emissions_factor_lb_PM25_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_facto
    can_serve_dhw::Bool = true # If ExistingBoiler can supply heat to the domesti
    can_serve_space_heating::Bool = true # If ExistingBoiler can supply heat to t
    can_serve_process_heat::Bool = true # If ExistingBoiler can supply heat to th
```

Max ExistingBoiler size

The maximum size [kW] of the `ExistingBoiler` will be set based on the peak heat demand as follows:

```
max_kw = max_heat_demand_kw * max_thermal_factor_on_peak_load
```

ExistingBoiler operating costs

The `ExistingBoiler`'s `fuel_cost_per_mmbtu` field is a required input. The `fuel_cost_per_mmbtu` can be a scalar, a list of 12 monthly values, or a time series of values for every time step.

Determining `efficiency`

Must supply either: `efficiency` or `production_type`.

If `efficiency` is not supplied, the `efficiency` will be determined based on the `production_type`. If `production_type` is not supplied, it defaults to `hot_water`. The following defaults are used:

```
existing_boiler_efficiency_defaults = Dict(
    "hot_water" => 0.8,
    "steam" => 0.75
)
```

CHP

`REopt.CHP` — Type

`CHP` is an optional REopt input with the following keys and default values:

```
    prime_mover::Union{String, Nothing} = nothing # Suggested to inform applicabl
    fuel_cost_per_mmbtu::Union{<:Real, AbstractVector{<:Real}} = [] # REQUIRED. C

    # Required "custom inputs" if not providing prime_mover:
    installed_cost_per_kw::Union{Float64, AbstractVector{Float64}} = NaN # Instal
    tech_sizes_for_cost_curve::Union{Float64, AbstractVector{Float64}} = NaN # Si
    om_cost_per_kwh::Float64 = NaN # CHP non-fuel variable operations and mainter
    electric_efficiency_full_load::Float64 = NaN # Electric efficiency of CHP pri
    electric_efficiency_half_load::Float64 = NaN # Electric efficiency of CHP pri
    min_turn_down_fraction::Float64 = NaN # Minimum CHP electric loading in fract
    thermal_efficiency_full_load::Float64 = NaN # CHP fraction of fuel energy cor
    thermal_efficiency_half_load::Float64 = NaN # CHP fraction of fuel energy cor
    min_allowable_kw::Float64 = NaN # Minimum CHP size (based on electric) that s
    cooling_thermal_factor::Float64 = NaN  # only needed with cooling load
    unavailability_periods::AbstractVector{Dict} = Dict[] # CHP unavailability pe
    unavailability_hourly::AbstractVector{Int64} = Int64[] # Hourly 8760 profile

    # Optional inputs:
    size_class::Union{Int, Nothing} = nothing # CHP size class for using appropri
    min_kw::Float64 = 0.0 # Minimum CHP size (based on electric) constraint for c
    max_kw::Float64 = NaN # Maximum CHP size (based on electric) constraint for c
```

```
    fuel_type::String = "natural_gas" # "restrict_to": ["natural_gas", "landfill_
    om_cost_per_kw::Float64 = 0.0 # Annual CHP fixed operations and maintenance c
    om_cost_per_hr_per_kw_rated::Float64 = 0.0 # CHP non-fuel variable operations
    supplementary_firing_capital_cost_per_kw::Float64 = 150.0 # Installed CHP sup
    supplementary_firing_max_steam_ratio::Float64 = 1.0 # Ratio of max fired stea
    supplementary_firing_efficiency::Float64 = 0.92 # Thermal efficiency of the i
    standby_rate_per_kw_per_month::Float64 = 0.0 # Standby rate charged to CHP ba
    reduces_demand_charges::Bool = true # Boolean indicator if CHP does not reduc
    can_supply_steam_turbine::Bool=false # If CHP can supply steam to the steam t
    can_serve_dhw::Bool = true # If CHP can supply heat to the domestic hot water
    can_serve_space_heating::Bool = true # If CHP can supply heat to the space he
    can_serve_process_heat::Bool = true # If CHP can supply heat to the process h
    is_electric_only::Bool = false # If CHP is a prime generator that does not su
    serve_absorption_chiller_only::Bool = false # If CHP produced heat either ser
    months_serving_absorption_chiller_only::AbstractVector{Int64} = Int64[] # mor
    follow_electrical_load::Bool = false # If CHP follows the electrical load by
    include_cooling_in_chp_size::Bool = false # If true, includes cooling load (v

    macrs_option_years::Int = 5 # Notes: this value cannot be 0 if aiming to appl
    macrs_bonus_fraction::Float64 = 1.0 #Note: default may change if Site.sector
    macrs_itc_reduction::Float64 = 0.5
    federal_itc_fraction::Float64 = 0.0
    federal_rebate_per_kw::Float64 = 0.0
    state_ibi_fraction::Float64 = 0.0
    state_ibi_max::Float64 = 1.0e10
    state_rebate_per_kw::Float64 = 0.0
    state_rebate_max::Float64 = 1.0e10
    utility_ibi_fraction::Float64 = 0.0
    utility_ibi_max::Float64 = 1.0e10
    utility_rebate_per_kw::Float64 = 0.0
    utility_rebate_max::Float64 = 1.0e10
    production_incentive_per_kwh::Float64 = 0.0 # revenue from production incenti
    production_incentive_max_benefit::Float64 = 1.0e9 # maximum allowable annual
    production_incentive_years::Int = 0 # number of year in which production ince
    production_incentive_max_kw::Float64 = 1.0e9 # maximum allowable system size
    can_net_meter::Bool = false
    can_wholesale::Bool = false
    can_export_beyond_nem_limit::Bool = false
    can_curtail::Bool = false
    fuel_renewable_energy_fraction::Float64 = FUEL_DEFAULTS["fuel_renewable_energ
```

```
    emissions_factor_lb_CO2_per_mmbtu::Float64 = FUEL_DEFAULTS["emissions_factor_
    emissions_factor_lb_NOx_per_mmbtu::Float64 = FUEL_DEFAULTS["emissions_factor_
    emissions_factor_lb_SO2_per_mmbtu::Float64 = FUEL_DEFAULTS["emissions_factor_
    emissions_factor_lb_PM25_per_mmbtu::Float64 = FUEL_DEFAULTS["emissions_facto
```

Defaults and required inputs

See the `get_chp_defaults_prime_mover_size_class()` function docstring for details on the logic of choosing the type of CHP that is modeled If no information is provided, the default `prime_mover` is `recip_engine` and the `size_class` is 0 which represents the widest range of sizes available.

`fuel_cost_per_mmbtu` is always required and can be a scalar, a list of 12 monthly values, or a time series of values for every time step

AbsorptionChiller

`REopt.AbsorptionChiller` — Type

`AbsorptionChiller` is an optional REopt input with the following keys and default values:

```
    thermal_consumption_hot_water_or_steam::Union{String, Nothing} = nothing  # D
    chp_prime_mover::String = ""  # Informs thermal_consumption_hot_water_or_stea

    # Defaults for fields below are dependent on thermal_consumption_hot_water_or
    installed_cost_per_ton::Union{Float64, Nothing} = nothing # Thermal power-bas
    om_cost_per_ton::Union{Float64, Nothing} = nothing # Yearly fixed O&M cost or
    min_ton::Float64 = 0.0, # Minimum thermal power size constraint for optimizat
    max_ton::Float64 = BIG_NUMBER, # Maximum thermal power size constraint for op
    cop_thermal::Union{Float64, Nothing} = nothing, # Absorption chiller system c
    cop_electric::Float64 = 14.1, # Absorption chiller electric consumption CoP f
    macrs_option_years::Float64 = 0, # MACRS schedule for financial analysis. Set
    macrs_bonus_fraction::Float64 = 0 # Percent of upfront project costs to depre
    heating_load_input::Union{String, Nothing} = nothing # heating load that serv
```

!!! Note To model AbsorptionChiller, there is logic which informs defaults for costs and COP: (i)

thermal_consumption_hot_water_or_steam from ["steam", "hot*water"], (ii) `chp*prime*mover` from ["recipengine", "micro*turbine", "combustion*turbine", "fuel*cell"], or (iii) if (i) and (ii) are not provided, the default `thermal*consumption*hot*water*or*steam is hot*water` The defaults for costs and COP will be populated from data/absorption*chiller/defaults.json, based on the thermal_consumption_hot_water_or_steam or chp_prime_mover. boiler_type is "steam" if prime_mover is "combustion*turbine" and is "hot*water" for all other chp_prime_mover types.

Boiler

REopt.Boiler — Type

```
Boiler
```

When modeling a heating load an ExistingBoiler model is created even if user does not provide the ExistingBoiler key. The Boiler model is not created by default. If a user provides the Boiler key then the optimal scenario has the option to purchase this new Boiler to meet the heating load in addition to using the ExistingBoiler to meet the heating load.

```
function Boiler(;
    min_mmbtu_per_hour::Real = 0.0, # Minimum thermal power size
    max_mmbtu_per_hour::Real = 0.0, # Maximum thermal power size
    efficiency::Real = 0.8, # boiler system efficiency - conversion of fuel to us
    fuel_cost_per_mmbtu::Union{<:Real, AbstractVector{<:Real}} = 0.0,
    macrs_option_years::Int = 0, # MACRS schedule for financial analysis. Set to
    macrs_bonus_fraction::Real = 0.0, # Fraction of upfront project costs to depr
    installed_cost_per_mmbtu_per_hour::Real = 293000.0, # Thermal power-based cos
    om_cost_per_mmbtu_per_hour::Real = 2930.0, # Thermal power-based fixed O&M co
    om_cost_per_mmbtu::Real = 0.0, # Thermal energy-based variable O&M cost
    fuel_type::String = "natural_gas",  # "restrict_to": ["natural_gas", "landfil
    can_supply_steam_turbine::Bool = true # If the boiler can supply steam to the
    can_serve_dhw::Bool = true # If Boiler can supply heat to the domestic hot wa
    can_serve_space_heating::Bool = true # If Boiler can supply heat to the space
    can_serve_process_heat::Bool = true # If Boiler can supply heat to the proces
    fuel_renewable_energy_fraction::Real = get(FUEL_DEFAULTS["fuel_renewable_ener
    emissions_factor_lb_CO2_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_factor
    emissions_factor_lb_NOx_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_factor
```

```
    emissions_factor_lb_SO2_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_factor
    emissions_factor_lb_PM25_per_mmbtu::Real = get(FUEL_DEFAULTS["emissions_facto
)
```

HotThermalStorage

REopt.HotThermalStorageDefaults — Type

`HotThermalStorage` is an optional REopt input with the following keys and default values:

```
    min_gal::Float64 = 0.0
    max_gal::Float64 = 0.0
    hot_water_temp_degF::Float64 = 180.0
    cool_water_temp_degF::Float64 = 160.0
    internal_efficiency_fraction::Float64 = 0.999999
    soc_min_fraction::Float64 = 0.1
    soc_init_fraction::Float64 = 0.5
    installed_cost_per_gal::Float64 = 1.50
    thermal_decay_rate_fraction::Float64 = 0.0004
    om_cost_per_gal::Float64 = 0.0
    macrs_option_years::Int = 5 #Note: default may change if Site.sector is not '
    macrs_bonus_fraction::Float64 = 1.0 #Note: default may change if Site.sector
    macrs_itc_reduction::Float64 = 0.5
    total_itc_fraction::Float64 = 0.3 #Note: default may change if Site.sector is
    total_rebate_per_kwh::Float64 = 0.0
    can_serve_dhw::Bool = true
    can_serve_space_heating::Bool = true
    can_serve_process_heat::Bool = false
```

HighTempThermalStorage

REopt.HighTempThermalStorageDefaults — Type

`HighTempThermalStorage` is an optional REopt input with the following keys and default values:

```
    fluid::String = "INCOMP::Nak"
    min_kwh::Float64 = 0.0
    max_kwh::Float64 = 0.0
    hot_temp_degF::Float64 = 1065.0
    cool_temp_degF::Float64 = 554.0
    internal_efficiency_fraction::Float64 = 0.999999
    soc_min_fraction::Float64 = 0.1
    soc_init_fraction::Float64 = 0.5
    installed_cost_per_kwh::Float64 = 86.0
    thermal_decay_rate_fraction::Float64 = 0.0004
    om_cost_per_kwh::Float64 = 0.0
    macrs_option_years::Int = 5 #Note: default may change if Site.sector is not '
    macrs_bonus_fraction::Float64 = 1.0 #Note: default may change if Site.sector
    macrs_itc_reduction::Float64 = 0.5
    total_itc_fraction::Float64 = 0.3 #Note: default may change if Site.sector is
    total_rebate_per_kwh::Float64 = 0.0
    can_supply_steam_turbine::Bool = true
    can_serve_dhw::Bool = false
    can_serve_space_heating:Bool = false
    can_serve_process_heat::Bool = true
    one_direction_flow::Bool = false
```

ColdThermalStorage

`REopt.ColdThermalStorageDefaults` — Type

Cold thermal energy storage sytem; specifically, a chilled water system used to meet thermal cooling loads.

`ColdThermalStorage` is an optional REopt input with the following keys and default values:

```
    min_gal::Float64 = 0.0
    max_gal::Float64 = 0.0
    hot_water_temp_degF::Float64 = 56.0 # Warmed-side return water temperature fr
    cool_water_temp_degF::Float64 = 44.0 # Chilled-side supply water temperature
    internal_efficiency_fraction::Float64 = 0.999999 # Thermal losses due to mixi
    soc_min_fraction::Float64 = 0.1 # Minimum allowable TES thermal state of char
```

```
    soc_init_fraction::Float64 = 0.5 # TES thermal state of charge at first hour
    installed_cost_per_gal::Float64 = 1.50 # Thermal energy-based cost of TES (e.
    thermal_decay_rate_fraction::Float64 = 0.0004 # Thermal loss (gain) rate as a
    om_cost_per_gal::Float64 = 0.0 # Yearly fixed O&M cost dependent on storage e
    macrs_option_years::Int = 5 #Note: default may change if Site.sector is not '
    macrs_bonus_fraction::Float64 = 1.0 #Note: default may change if Site.sector
    macrs_itc_reduction::Float64 = 0.5
    total_itc_fraction::Float64 = 0.3 #Note: default may change if Site.sector is
    total_rebate_per_kwh::Float64 = 0.0
```

HeatingLoad

REopt.HeatingLoad — Method

`HeatingLoad` is a base function for the types of heating load inputs with the following keys and default values:

```
    load_type::String = "",  # Valid options are space_heating for SpaceHeatingLo
    doe_reference_name::String = "",  # For SpaceHeatingLoad and DomesticHotWater
    blended_doe_reference_names::Array{String, 1} = String[],  # For SpaceHeating
    blended_doe_reference_percents::Array{<:Real,1} = Real[],  # For SpaceHeating
    industrial_reference_name::String = "",  # For ProcessHeatLoad
    blended_industrial_reference_names::Array{String, 1} = String[],  # For Proce
    blended_industrial_reference_percents::Array{<:Real,1} = Real[],  # For Proce
    city::String = "",
    year::Union{Int, Nothing} = doe_reference_name ≠ "" || blended_doe_reference_
    annual_mmbtu::Union{Real, Nothing} = nothing,
    monthly_mmbtu::Array{<:Real,1} = Real[],
    addressable_load_fraction::Any = 1.0,  # Fraction of input fuel load which is
    fuel_loads_mmbtu_per_hour::Array{<:Real,1} = Real[], # Vector of space heatin
    normalize_and_scale_load_profile_input::Bool = false,  # Takes fuel_loads_mmb
    existing_boiler_efficiency::Real = NaN
```

There are different ways to define a heating load:

1. A time-series via the `fuel_loads_mmbtu_per_hour`,
2. Scaling a DOE Commercial Reference Building (CRB) or industrial reference profile or a blend

of profiles to either the `annual_mmbtu` or `monthly_mmbtu` values;

3. Using the same `doe_reference_name` or `blended_doe_reference_names` from the `ElectricLoad`.
4. A time-series via the `fuel_loads_mmbtu_per_hour` along with `annual_mmbtu` or `monthly_mmbtu` with `normalize_and_scale_load_profile_input=true`

When using an `ElectricLoad` defined from a `doe_reference_name` or `blended_doe_reference_names` one only needs to provide an empty Dict in the scenario JSON to add a `SpaceHeatingLoad` to a `Scenario`, i.e.:

```
...
"ElectricLoad": {"doe_reference_name": "MidriseApartment"},
"SpaceHeatingLoad" : {},
...
```

In this case the values provided for `doe_reference_name`, or `blended_doe_reference_names` and `blended_doe_reference_percents` are copied from the `ElectricLoad` to the the particular `HeatingLoad` type.

!!! note for all heating loads Hot water, space heating, and process heat "load" inputs are in terms of energy input required (boiler fuel), not the actual end use thermal energy demand. The fuel energy is multiplied by the existing*boiler*efficiency to get the actual energy demand.

CoolingLoad

`REopt.CoolingLoad` — Type

`CoolingLoad` is an optional REopt input with the following keys and default values:

```
    doe_reference_name::String = "",
    blended_doe_reference_names::Array{String, 1} = String[],
    blended_doe_reference_percents::Array{<:Real,1} = Real[],
    city::String = "",
    year::Int = doe_reference_name ≠ "" || blended_doe_reference_names ≠ String[]
    annual_tonhour::Union{Real, Nothing} = nothing,
    monthly_tonhour::Array{<:Real,1} = Real[],
```

```
    thermal_loads_ton::Array{<:Real,1} = Real[], # Vector of cooling thermal load
    annual_fraction_of_electric_load::Union{Real, Nothing} = nothing, # Fraction
    monthly_fractions_of_electric_load::Array{<:Real,1} = Real[],
    per_time_step_fractions_of_electric_load::Array{<:Real,1} = Real[]
```

There are many ways to define a `CoolingLoad`:

1. a time-series via the `thermal_loads_ton`,
2. DoE Commercial Reference Building (CRB) profile or a blend of CRB profiles which uses the buildings' fraction of total electric for cooling profile applied to the `ElectricLoad`
3. scaling a DoE Commercial Reference Building (CRB) profile or a blend of CRB profiles using `annual_tonhour` or `monthly_tonhour`
4. the `annual_fraction_of_electric_load`, `monthly_fractions_of_electric_load`, or `per_time_step_fractions_of_electric_load` values, which get applied to the `ElectricLoad` to determine the cooling electric load;
5. or using the `doe_reference_name` or `blended_doe_reference_names` from the `ElectricLoad`.

The electric-based `loads_kw` of the `CoolingLoad` is a *subset* of the total electric load `ElectricLoad`, so `CoolingLoad.loads_kw` for the BAU/conventional electric consumption of the `existing_chiller` is subtracted from the `ElectricLoad` for the non-cooling electric load balance constraint in the model.

When using an `ElectricLoad` defined from a `doe_reference_name` or `blended_doe_reference_names` one only needs to provide an empty Dict in the scenario JSON to add a `CoolingLoad` to a `Scenario`, i.e.:

```
...
"ElectricLoad": {"doe_reference_name": "MidriseApartment"},
"CoolingLoad" : {},
...
```

In this case the values provided for `doe_reference_name`, or `blended_doe_reference_names` and `blended_doe_reference_percents` are copied from the `ElectricLoad` to the `CoolingLoad`.

FlexibleHVAC

REopt.FlexibleHVAC — Type

FlexibleHVAC is an optional REopt input with the following keys and default values:

```
    system_matrix::AbstractMatrix{Float64}  # N x N, with N states (temperatures
    input_matrix::AbstractMatrix{Float64}  # N x M, with M inputs
    exogenous_inputs::AbstractMatrix{Float64}  # M x T, with T time steps
    control_node::Int64
    initial_temperatures::AbstractVector{Float64}
    temperature_upper_bound_degC::Union{Real, Nothing}
    temperature_lower_bound_degC::Union{Real, Nothing}
    installed_cost::Float64
```

Every model with FlexibleHVAC includes a preprocessing step to calculate the business-as-usual (BAU) cost of meeting the thermal loads using a dead-band controller. The BAU cost is then used in the binary decision for purchasing the FlexibleHVAC system: if the FlexibleHVAC system is purchased then the heating and cooling costs are determined by the HVAC dispatch that minimizes the lifecycle cost of energy. If the FlexibleHVAC system is not purchased then the BAU heating and cooling costs must be paid.

There are two construction methods for FlexibleHVAC, which depend on whether or not the data was loaded in from a JSON file. The issue with data from JSON is that the vector-of-vectors from the JSON file must be appropriately converted to Julia Matrices. When loading in a Scenario from JSON that includes a FlexibleHVAC model, if you include the flex_hvac_from_json argument to the Scenario constructor then the conversion to Matrices will be done appropriately.

Note

At least one of the inputs for temperature_upper_bound_degC or temperature_lower_bound_degC must be provided to evaluate the FlexibleHVAC option. For example, if only temperature_lower_bound_degC is provided then only a heating system will be evaluated. Also, the heating system will only be used (or purchased) if the exogenous_inputs lead to the temperature at the control_node going below the temperature_lower_bound_degC.

Note

The `ExistingChiller` is electric and so its operating cost is determined by the `ElectricTariff`.

Note

BAU heating costs will be determined by the `ExistingBoiler` inputs, including `fuel_cost_per_mmbtu`.

ExistingChiller

`REopt.ExistingChiller` — Type

`ExistingChiller` is an optional REopt input with the following keys and default values:

```
    loads_kw_thermal::Vector{<:Real},
    cop::Union{Real, Nothing} = nothing,
    max_thermal_factor_on_peak_load::Real=1.25
    installed_cost_per_ton::Real = 0.0  # Represents needed CapEx in BAU, assumir
    installed_cost_dollars::Real = 0.0  # Represents needed CapEx in BAU, assumir
    retire_in_optimal::Bool = false  # Do NOT use in the optimal case (still usec
```

Max ExistingChiller size

The maximum size [kW] of the `ExistingChiller` will be set based on the peak thermal load as follows, and this is really the **actual** estimated size of the existing chiller at the site:

```
max_kw = maximum(loads_kw_thermal) * max_thermal_factor_on_peak_load
```

GHP

REopt.GHP — Type

GHP evaluations typically require the `GhpGhx.jl` package to be loaded unless the `GhpGhx.jl` package was already used externally to create `inputs_dict["GHP"]["ghpghx_responses"]`. See the Home page under "Additional package loading for GHP" for instructions. This `GHP` struct uses the response from `GhpGhx.jl` to process input parameters for REopt including additional cost parameters for `GHP`.

```
GHP
```

struct with outer constructor:

```
    require_ghp_purchase::Union{Bool, Int64} = false  # 0 = false, 1 = true
    installed_cost_heatpump_per_ton::Float64 = 1075.0
    installed_cost_wwhp_heating_pump_per_ton::Float64 = 700.0
    installed_cost_wwhp_cooling_pump_per_ton::Float64 = 700.0
    heatpump_capacity_sizing_factor_on_peak_load::Float64 = 1.1
    installed_cost_ghx_per_ft::Float64 = 14.0
    installed_cost_building_hydronic_loop_per_sqft = 1.70
    om_cost_per_sqft_year::Float64 = -0.51
    building_sqft::Float64                                  # Required input
    space_heating_efficiency_thermal_factor::Float64 = NaN  # Default depends on
    cooling_efficiency_thermal_factor::Float64 = NaN        # Default depends on
    ghpghx_response::Dict = Dict()
    can_serve_dhw::Bool = false
    max_ton::Real                                           # Maximum heat pump c
    max_number_of_boreholes::Real                           # Maximum GHX size
    load_served_by_ghp::String                              # "scaled" or "nonpea

    macrs_option_years::Int = 0
    macrs_bonus_fraction::Float64 = 0.0
    macrs_itc_reduction::Float64 = 0.5
    federal_itc_fraction::Float64 = 0.3 #Note: default may change if Site.sector
    federal_rebate_per_ton::Float64 = 0.0
    federal_rebate_per_kw::Float64 = 0.0
    state_ibi_fraction::Float64 = 0.0
    state_ibi_max::Float64 = 1.0e10
    state_rebate_per_ton::Float64 = 0.0
```

```
    state_rebate_per_kw::Float64 = 0.0
    state_rebate_max::Float64 = 1.0e10
    utility_ibi_fraction::Float64 = 0.0
    utility_ibi_max::Float64 = 1.0e10
    utility_rebate_per_ton::Float64 = 0.0
    utility_rebate_per_kw::Float64 = 0.0
    utility_rebate_max::Float64 = 1.0e10

    # Processed data from inputs and results of GhpGhx.jl
    heating_thermal_kw::Vector{Float64} = []
    cooling_thermal_kw::Vector{Float64} = []
    yearly_electric_consumption_kw::Vector{Float64} = []
    peak_combined_heatpump_thermal_ton::Float64 = NaN

    # Intermediate parameters for cost processing
    tech_sizes_for_cost_curve::Union{Float64, AbstractVector{Float64}} = NaN
    installed_cost_per_kw::Union{Float64, AbstractVector{Float64}} = NaN
    heatpump_capacity_ton::Float64 = NaN

    # Process and populate these parameters needed more directly by the model
    installed_cost::Float64 = NaN
    om_cost_year_one::Float64 = NaN
```

SteamTurbine

`REopt.SteamTurbine` — Type

`SteamTurbine` is an optional REopt input with the following keys and default values:

```
    size_class::Union{Int64, Nothing} = nothing
    min_kw::Float64 = 0.0
    max_kw::Float64 = 1.0e9
    electric_produced_to_thermal_consumed_ratio::Float64 = NaN
    thermal_produced_to_thermal_consumed_ratio::Float64 = NaN
    is_condensing::Bool = false
    inlet_steam_pressure_psig::Float64 = NaN
    inlet_steam_temperature_degF::Float64 = NaN
```

```
    inlet_steam_superheat_degF::Float64 = 0.0
    outlet_steam_pressure_psig::Float64 = NaN
    outlet_steam_min_vapor_fraction::Float64 = 0.8  # Minimum practical vapor fra
    isentropic_efficiency::Float64 = NaN
    gearbox_generator_efficiency::Float64 = NaN  # Combined gearbox (if applicabl
    net_to_gross_electric_ratio::Float64 = NaN  # Efficiency factor to account fo
    installed_cost_per_kw::Float64 = NaN   # Installed cost based on electric pow
    om_cost_per_kw::Float64 = 0.0  # Fixed O&M cost based on electric power capac
    om_cost_per_kwh::Float64 = NaN  # Variable O&M based on electric energy produ
    production_incentive_per_kwh::Float64 = 0.0 # revenue from production incenti
    production_incentive_max_benefit::Float64 = 1.0e9 # maximum allowable annual
    production_incentive_years::Int = 0 # number of year in which production ince
    production_incentive_max_kw::Float64 = 1.0e9 # maximum allowable system size

    can_net_meter::Bool = false
    can_wholesale::Bool = false
    can_export_beyond_nem_limit::Bool = false
    can_curtail::Bool = false
    can_waste_heat::Bool = false
    can_serve_dhw::Bool = true
    can_serve_space_heating::Bool = true
    can_serve_process_heat::Bool = true
    charge_storage_only::Bool = false

    macrs_option_years::Int = 5 # Note that this value cannot be 0 if aiming to a
    macrs_bonus_fraction::Float64 = 1.0
```

ElectricHeater

`REopt.ElectricHeater` — Type

ElectricHeater

If a user provides the `ElectricHeater` key then the optimal scenario has the option to purchase this new `ElectricHeater` to meet the heating load in addition to using the `ExistingBoiler` to meet the heating load.

```
function ElectricHeater(;
    min_mmbtu_per_hour::Real = 0.0, # Minimum thermal power size
    max_mmbtu_per_hour::Real = BIG_NUMBER, # Maximum thermal power size
    installed_cost_per_mmbtu_per_hour::Union{Real, nothing} = nothing, # Thermal
    om_cost_per_mmbtu_per_hour::Union{Real, nothing} = nothing, # Thermal power-b
    macrs_option_years::Int = 0, # MACRS schedule for financial analysis. Set to
    macrs_bonus_fraction::Real = 0.0, # Fraction of upfront project costs to depr
    can_supply_steam_turbine::Union{Bool, nothing} = nothing # If the boiler can
    cop::Union{Real, nothing} = nothing # COP of the heating (i.e., thermal produ
    can_serve_dhw::Bool = true # If electric heater can supply heat to the domest
    can_serve_space_heating::Bool = true # If electric heater can supply heat to
    can_serve_process_heat::Bool = true # If electric heater can supply heat to t
)
```

ASHPSpaceHeater

REopt.ASHPSpaceHeater — Function

ASHPSpaceHeater

If a user provides the `ASHPSpaceHeater` key then the optimal scenario has the option to purchase this new `ASHPSpaceHeater` to meet the heating load in addition to using the `ExistingBoiler` to meet the heating load.

```
function ASHPSpaceHeater(;
    min_ton::Real = 0.0, # Minimum thermal power size
    max_ton::Real = BIG_NUMBER, # Maximum thermal power size
    min_allowable_ton::Union{Real, Nothing} = nothing, # Minimum nonzero thermal
    min_allowable_peak_capacity_fraction::Union{Real, Nothing} = nothing, # minim
    sizing_factor::::Union{Real, Nothing} = nothing, # Size multiplier of system,
    om_cost_per_ton::Union{Real, nothing} = nothing, # Thermal power-based fixed
    macrs_option_years::Int = 0, # MACRS schedule for financial analysis. Set to
    macrs_bonus_fraction::Real = 0.0, # Fraction of upfront project costs to depr
    can_serve_cooling::Union{Bool, Nothing} = nothing # If ASHP can supply heat t
    force_into_system::Bool = false # force into system to serve all space heatin
    force_dispatch::Bool = true # force ASHP to meet load or maximize output if t
    avoided_capex_by_ashp_present_value::Real = 0.0 # avoided capital expenditure
```

```
    #The following inputs are used to create the attributes heating_cop and heati
    heating_cop_reference::Array{<:Real,1}, # COP of the heating (i.e., thermal p
    heating_cf_reference::Array{<:Real,1}, # ASHP's heating capacity factor curve
    heating_reference_temps_degF ::Array{<:Real,1}, # ASHP's reference temperatur
    back_up_temp_threshold_degF::Real = 10, # Degree in F that system switches fr

    #The following inputs are used to create the attributes cooling_cop and cooli
    cooling_cop::Array{<:Real,1}, # COP of the cooling (i.e., thermal produced /
    cooling_cf::Array{<:Real,1}, # ASHP's cooling capacity factor curves
    cooling_reference_temps_degF ::Array{<:Real,1}, # ASHP's reference temperatur

    #The following inputs are taken from the Site object:
    ambient_temp_degF::Array{<:Real,1}  #time series of ambient temperature
    heating_load::Array{Real,1} # time series of site space heating load
    cooling_load::Union{Array{Real,1}, Nothing} # time series of site cooling loa
)
```

ASHPWaterHeater

REopt.ASHPWaterHeater — Function

ASHPWaterHeater

If a user provides the `ASHPWaterHeater` key then the optimal scenario has the option to purchase this new `ASHPWaterHeater` to meet the domestic hot water load in addition to using the `ExistingBoiler` to meet the domestic hot water load.

```
function ASHPWaterHeater(;
    min_ton::Real = 0.0, # Minimum thermal power size
    max_ton::Real = BIG_NUMBER, # Maximum thermal power size
    min_allowable_ton::Real = 0.0 # Minimum nonzero thermal power size if include
    installed_cost_per_ton::Union{Real, nothing} = nothing, # Thermal power-based
    om_cost_per_ton::Union{Real, nothing} = nothing, # Thermal power-based fixed
    macrs_option_years::Int = 0, # MACRS schedule for financial analysis. Set to
    macrs_bonus_fraction::Real = 0.0, # Fraction of upfront project costs to depr
    can_supply_steam_turbine::Union{Bool, nothing} = nothing # If the boiler can
```

```
    avoided_capex_by_ashp_present_value::Real = 0.0 # avoided capital expenditure
    force_into_system::Bool = false # force into system to serve all hot water lo
    force_dispatch::Bool = true # force ASHP to meet load or maximize output if t

    #The following inputs are used to create the attributes heating_cop and heati
    heating_cop_reference::Array{<:Real,1}, # COP of the heating (i.e., thermal p
    heating_cf_reference::Array{<:Real,1}, # ASHP's heating capacity factor curve
    heating_reference_temps_degF ::Array{<:Real,1}, # ASHP's reference temperatur
    back_up_temp_threshold_degF::Real = 10 # temperature threshold at which backu

    #The following inputs are taken from the Site object:
    ambient_temp_degF::Array{<:Real,1} = Float64[] # time series of ambient tempe
    heating_load::Array{<:Real,1} # time series of site domestic hot water load
)
```

CST

REopt.CST — Type

```
CST
```

If a user provides the `CST` key then the optimal scenario has the option to purchase this new `CST` technology to meet compatible heating loads in addition to using the `ExistingBoiler` to meet the heating load(s).

```
function CST(;
    min_kw::Real = 0.0, # Minimum thermal power size (capacity)
    max_kw::Real = BIG_NUMBER, # Maximum thermal power size (capacity)
    production_factor::AbstractVector{<:Real} = Float64[],  production factor
    elec_consumption_factor_series::AbstractVector{<:Real} = Float64[], electric
    acres_per_kw::Union{Real,Nothing} = nothing, #
    macrs_option_years::Union{Int,Nothing} = nothing, # MACRS schedule for financ
    macrs_bonus_fraction::Union{Real,Nothing} = nothing, # Fraction of upfront pr
    installed_cost_per_kw::Union{Real,Nothing} = nothing, # Thermal power-based c
    om_cost_per_kw::Union{Real,Nothing} = nothing, # Thermal power-based fixed O&
    om_cost_per_kwh::Union{Real,Nothing} = nothing, # Thermal energy produced-bas
    tech_type::Union{String,Nothing} = nothing,  # restrict to: ["ptc", "mst", "l
```

```
    can_supply_steam_turbine::Union{Bool,Nothing} = nothing # If the CST can supp
    can_serve_dhw::Union{Bool,Nothing} = nothing # If CST can supply heat to the
    can_serve_space_heating::Union{Bool,Nothing} = nothing # If CST can supply he
    can_serve_process_heat::Union{Bool,Nothing} = nothing # If CST can supply hea
    can_waste_heat::Union{Bool,Nothing} = nothing # If CST can curtail excess hea
    charge_storage_only::Union{Bool,Nothing} = nothing # If CST can only supply h
    emissions_factor_lb_CO2_per_mmbtu::Real = 0.0
    emissions_factor_lb_NOx_per_mmbtu::Real = 0.0
    emissions_factor_lb_SO2_per_mmbtu::Real = 0.0
    emissions_factor_lb_PM25_per_mmbtu::Real = 0.0
    inlet_temp_degF::Real = 400.0 # heat transfer medium temperature at inlet of
    outlet_temp_degF::Real = 70.0 # heat transfer medium temperature at outlet of
)
```

« Examples Outputs »

Powered by Documenter.jl and the Julia Programming Language.